ASSIGNMENT 8: OPTIMIZING DELIVERY ROUTES FOR A LOGISTICS COMPANY (TSP: LEAST-COST BRANCH AND BOUND)

T.Y. B.TECH | **Year: 2026–27** | **Semester: V**

Department: Information Technology | **Division: A** | **Course Code: BIT25PC04**

Course: Design Analysis and Algorithm Laboratory

NAME	Sarvam Tarapure
PRN	124B1F048

GitHub Repository: https://github.com/Black-sys-dot/Design-and-Analysis-of-Algorithms

Problem Statement

SwiftShip, a logistics company, must plan a delivery tour that visits every city exactly once and returns to its starting city. The aim is to minimize total route cost, considering varying travel costs and delivery time constraints.

Checking all possible tours requires about (N–1)! permutations, which quickly becomes impractical as the number of cities grows. The task is to implement Least-Cost (LC) Branch and Bound, using lower bounds to avoid exploring partial routes that cannot improve the best known tour.

Course objectives

- Understand computational complexity of algorithms.
- Choose appropriate algorithm-design strategies for practical problems.

Course outcomes

- Apply Branch and Bound to solve problems.
- Classify P, NP, NP-complete and NP-hard problems.

Theory and Cost-Matrix Reduction

1. Travelling Salesman Problem

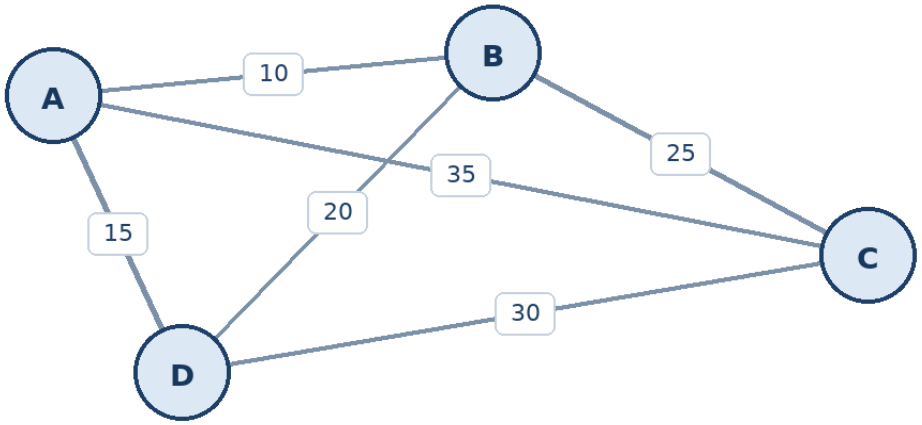
The Travelling Salesman Problem (TSP) asks for a minimum-cost cycle that visits every city once and returns to the starting city. Each city is a vertex, and travel costs are stored in a matrix. Diagonal entries are set to infinity because a city cannot be used as a move to itself.

2. Least-Cost Branch and Bound

Branch and Bound builds a search tree of partial tours. Each node stores the partial path, accumulated route cost, and a lower-bound estimate for completing the tour. A min-priority queue expands the node with the smallest bound first. If a node cannot beat the best complete tour found so far, it is pruned.

3. Lower bound through matrix reduction

Set forbidden moves to ∞. For each row, subtract its finite minimum from all finite entries, then do the same for each column. The sum of the subtracted minima contributes to a lower bound on the remaining tour cost. The reduced matrix helps estimate promising branches before fully exploring them.



Illustrative weighted graph: vertices are cities; edge labels are travel costs.

Illustrative weighted city graph. Edge labels represent travel costs; the optimal tour depends on the complete cost matrix.

4. Search strategy

Reduce the initial matrix and create the root node with its initial bound. Repeatedly remove the lowest-bound node from the priority queue. For each unvisited city, form a child by selecting the next edge, blocking the used row and column, preventing an immediate return to the start, and reducing the resulting matrix. Push the child only when its bound is below the best known tour cost.

Algorithm, Pseudocode and Complexity

Least-Cost Branch and Bound procedure

```
TSP_BranchAndBound(costMatrix, start):
    reducedMatrix, lb0 = reduceMatrix(costMatrix)
    root = Node(matrix=reducedMatrix,
                bound=lb0, path=[start], level=0)
    bestCost = INF
    bestPath = null
    PQ = min-heap ordered by bound
    PQ.push(root)

    while PQ is not empty:
        node = PQ.popMin()
        if node.bound >= bestCost:
            continue                    // prune

        if node.level == N - 1:
            last = node.path.back()
            total = original path cost
                    + originalCost[last][start]
            if total < bestCost:
                bestCost = total
                bestPath = node.path + [start]
            continue

        u = node.path.back()
        for each city v not in node.path:
            childMatrix = copy(node.matrix)
            block row u and column v
            block edge v -> start when required
            childBound = node.bound
                         + originalCost[u][v]
                         + reduceMatrix(childMatrix)
            if childBound < bestCost:
                PQ.push(Node(childMatrix, childBound,
                             node.path + [v], node.level + 1))

    return bestCost, bestPath
```

Implementation note

The lower bound and accumulated route cost must be tracked consistently. The completed tour cost should be computed from the original edge costs along the path plus the final edge back to the start; a reduced-matrix bound should not be mistaken for the actual tour cost.

Complexity analysis

- Worst-case time remains factorial, O(N!), because pruning may be weak on some inputs.
- Matrix copying and reduction cost $O(N^2)$ per expanded node in a straightforward implementation.
- The priority queue can consume substantial memory when many nodes remain promising.
- The source assignment estimates around 12–16 cities as a practical range, depending on pruning and hardware.

Pruning, Constraints and Conclusion

Pruning rule

Let bestCost be the cost of the best complete tour found so far. If a partial node has a valid lower bound greater than or equal to bestCost, it cannot lead to a better solution and may be discarded. Best-first expansion prioritizes promising nodes but still requires a valid lower bound.

Handling delivery time constraints

Time constraints can be incorporated by adding penalties for late arrivals or by rejecting partial paths that already exceed a time window. The bound must remain valid for the constrained problem; otherwise pruning could incorrectly remove an optimal feasible tour.

Example workflow

1. Build the N × N cost matrix from travel/fuel costs and set diagonal entries to ∞.
2. Reduce the matrix and store the initial lower bound.
3. Expand the partial tour with the smallest bound.
4. Generate children for unvisited cities, update their matrices and bounds, and prune unpromising branches.
5. When a full tour is completed, add the return-to-start cost and update the best solution.
6. Continue until no better candidate remains.

Conclusion

Least-Cost Branch and Bound solves TSP exactly by combining a search tree with lower bounds and best-first expansion. Matrix reduction can eliminate many branches compared with exhaustive permutation search, while worst-case running time remains factorial. It is appropriate when an exact answer is needed for a moderate number of cities. For larger or real-time routing tasks, exact search can be combined with 2-opt, Christofides or metaheuristics, as noted in the source assignment.

End of Assignment 8